

Google Summer of Code 2026 Project Proposal: Rust Foundation

1. Project Title

ZKS Protocol Decentralized Onion Routing with Nostr

2. Personal Information

- **Name:** Md. Wasif Faisal
- **Email:** md.wasif.faisal@g.bracu.ac.bd
- **University/Study Programme:** BSc in Computer Science, BRAC University
- **Time Zone:** Dhaka (UTC+6)
- **GitHub Profile:** <https://github.com/cswasif>
- **Portfolio/Website:** <https://github.com/zks-protocol>
- **Rust Experience:**
 - Creator and lead developer of the ZKS Protocol, an advanced research prototype for a post-quantum secure onion routing network built entirely in Rust.
 - Deep understanding of Rust's async ecosystem (Tokio), memory safety guarantees, and decentralized peer-to-peer networking (libp2p).
- **Open-Source Experience:**
 - Authored the `zks_sdk`, `zks_crypto`, and `zks_pqcrypto` crates currently published on crates.io.
 - [List any other contributions to Rust or other open-source projects]

3. Project Information

Abstract

The ZKS (Zero Knowledge Swarm) Protocol is a post-quantum secure onion routing network built entirely in Rust. While the core protocol is currently an advanced research prototype featuring 256-bit post-quantum encryption (NIST Level 5 via ML-KEM-1024), it relies on a centralized Cloudflare signaling server for peer discovery. This GSoC project aims to transition ZKS Protocol into a fully decentralized, community-driven network by integrating the Nostr protocol as a global, permissionless directory layer. By combining Nostr's massive decentralized relay infrastructure with ZKS's ephemeral "burner" identities and mathematical anonymity, this project will provide the Rust community with a quantum-safe, serverless alternative to the Tor network.

Detailed Description

The Rust community lacks a native, post-quantum secure onion routing library that developers can seamlessly embed into decentralized applications without relying on centralized infrastructure. The generic `zks://` protocol provides multi-hop anonymous connections, but its current reliance on a single Cloudflare Worker for peer coordination presents a centralized point of failure and creates infrastructure costs.

This project will mature the ZKS Protocol crates for the Rust ecosystem by focusing on two main pillars during the GSoC period:

1. **Decentralized Signaling via Nostr Integration:**

- Deprecate the existing `cloudflare_signaling.rs` implementation and replace it with `nostr_signaling.rs` using the Rust `nostr-sdk`.
- Leverage standard Nostr Replaceable Events (Kind 30000) for global Peer Discovery (advertising Guard, Middle, and Exit nodes).
- Implement "Burner" identity abstractions: ensuring ZKS nodes automatically generate and discard ephemeral `secp256k1` Nostr keypairs for every session query to prevent identity correlation and metadata leakage on public relays.

2. Performance & Circuit Optimization:

- Overhaul the `FaisalSwarmManager` to gracefully handle public Nostr relay rate-limits and connectivity drops.
- Introduce background circuit pre-building to reduce end-to-end connection latency from ~8.2s down to <2s, masking the slower discovery times of public Nostr relays.

Utility to the Rust Community: This project directly benefits the Rust ecosystem by providing a highly secure, memory-safe, and serverless networking stack. It aligns perfectly with Rust's goals of empowering everyone to build reliable software. By making quantum-safe, decentralized anonymity accessible through simple `Cargo.toml` dependencies, we enable Rust developers to effortlessly build censorship-resistant applications, secure messengers, and distributed privacy tools.

Project Size

- **Medium (~175 hours)**

4. Timeline and Deliverables (12-Week Plan)

Community Bonding Period (May)

- Engage with mentors to refine the architecture for the Nostr signaling transition and review the `nostr-sdk` capabilities.
- Introduce the "Burner Identity" state machine design to the broader Rust Zulip `#gsoc` community for feedback.

Week 1-3: Nostr Signaling Integration

- Implement `nostr_signaling.rs` to satisfy the existing `SignalingClientTrait`.
- Map the Swarm `Join` and `Discover` functions to Nostr `REQ` streams and `EVENT` broadcasts using Parameterized Replaceable Events (Kind 30000).

Week 4-6: Anonymity & Burner Identities

- Implement ephemeral key generation inside the `NostrSignalingClient` to ensure clients never reuse public keys when querying public relays.
- Add event expiry tags (`[ "expiration", "<timestamp>" ]`) so Nostr relays automatically drop disconnected ZKS nodes.
- **Midterm Milestone:** Demonstrate a fully functional, 3-hop post-quantum ZKS circuit bootstrapped entirely over a public Nostr relay (e.g., `wss://relay.damus.io`).

Week 7-9: Circuit Optimization & Latency Reduction

- Implement background circuit pre-building in `FaisalSwarmManager` to drastically reduce connection latency.
- Refactor peer health scoring to handle the noisy environment of a permissionless Nostr-based swarm.

Week 10: Local Randomness Transition

- Refactor the swarm manager to fetch cryptographic entropy (drand) natively via HTTP, replacing the legacy Cloudflare entropy oracle.
- Enforce `TrueEntropyRng` local fallbacks for high-assurance environments.

Week 11: Code Freeze & Documentation

- **Code Freeze:** No new features.
- Polish API documentation (rustdoc) for all updated crates, ensuring they meet the high standards of the Rust community.

Week 12: Final Polish & Submission

- Ensure 100% test coverage for the new Nostr networking modules.
- Finalize the GSoC project report and prepare the final presentation/demo.

5. Other Commitments

- I have no other major conflicting commitments (like heavy exams, long vacations, or full-time jobs) during the May - August period. I am fully committed to dedicating the required ~15-20 hours per week to ensure this GSoC project is a complete success.

6. Pre-proposal Contributions

- As the creator of the ZKS Protocol, I have authored the entirety of the existing codebase. I will continue to maintain and improve the project, addressing issues and merging PRs prior to the start of the GSoC period.